German University in Cairo
Media Engineering and Technology
CSEN/CSIS402: Computer Organization and Systems Programming

Project Milestone 2 Report

Control Unit Timing, RTLs, Control Signals, and Memory Contents

Name	Tutorial	ID
Habiba Abdou Hussein	T-6	64-1442
Baraa Khaled Mohamed	T-6	64-12893
Hady Weaam	T-7	61-13384
Ahmed Hagras	T-8	64-8836
Hamza Mohamed Abdelghany	T-8	64-13596
Amr Elsayed Elsayed	T-23	58-13562

Submission Date: May 18, 2026

1. Objective

This report documents the complete control unit design and timing developed for Milestone 2 of the basic computer project. The control unit is designed to support exactly the seven required instructions: XOR, ADD, LDA, STA, BUN, DIV, and ISZ. All unused control signals are held at logic zero to ensure a stable datapath. The design omits indirect addressing and interrupt handling as per project specifications, using only direct memory references and the standard instruction cycle.

2. Instruction Format and Opcode Decoder

The instruction format for this milestone utilises a 16-bit word structured as follows: bits [15:12] contain the opcode, bits [11:4] contain the address field, and bits [3:0] are unused and ignored. Consequently, the effective address is extracted from IR[11:4]. Per project requirements, the opcodes of the unused AND and BSA instructions from the standard basic computer have been reassigned to XOR and DIV respectively. Let D_0 through D_6 denote the decoded outputs of the 4-to-16 opcode decoder (only outputs 0–6 are utilised).

Decoder Output	Opcode [15:12]	Instruction	Operation	Meaning
D_0	0000	XOR	AC ← AC XOR M[AR]	Bitwise XOR with memory operand
D_1	0001	ADD	AC ← AC + M[AR]	Add memory operand to AC
D_2	0010	LDA	AC ← M[AR]	Load memory operand into AC
D_3	0011	STA	M[AR] ← AC	Store AC to memory
D_4	0100	BUN	PC ← AR	Branch unconditionally
D_5	0101	DIV	AC ← AC / M[AR]	Integer division
D_6	0110	ISZ	M[AR] ← M[AR]+1; if result=0, PC ← PC+1	Increment and skip if zero

3. RAM Contents

The RAM is loaded with the machine-code program and initialised data variables. The assembly program implements a for-loop that iterates three times, computing A += A XOR B and C += A / B each iteration. The variables A, B, C, and i are stored at hexadecimal addresses 0B, 0C, 0D, and 0E respectively. The loop label LOP corresponds to address 0x0000.

Address (hex)	Hex Value	Assembly / Meaning	Comment
0	20B0	LDA A	Load variable A into AC
1	00C0	XOR B	AC ← AC XOR B
2	10B0	ADD A	AC ← AC + A (A XOR B + A)
3	30B0	STA A	Store result back to A
4	20B0	LDA A	Reload updated A
5	50C0	DIV B	AC ← AC / B (integer division)

Address (hex)	Hex Value	Assembly / Meaning	Comment
6	10D0	ADD C	AC ← AC + C
7	30D0	STA C	Store result back to C
8	60E0	ISZ i	Increment loop counter i; skip if zero
9	4000	BUN LOP	Branch unconditionally to LOP (addr 0)
A	0000	HLT / unused	Halt or unused location
B	000F	A = 15	Initial value of A (decimal 15)
C	0005	B = 5	Initial value of B (decimal 5)
D	0000	C = 0	Initial value of C (decimal 0)
E	FFFD	i = –3	Initial loop counter (2's complement of –3)

4. Initial Register Values After Reset

Upon system reset, all programmable registers and the sequence counter are cleared to zero. The Program Counter (PC) begins execution at address 0x0000, which corresponds to the LOP label.

Register	Initial Value (hex)	Width
PC	0000	16 bits
AC	0000	16 bits
AR	0000	8 bits
DR	0000	16 bits
IR	0000	16 bits
TR	0000	16 bits
SC	000	3 bits (counts T0–T6)

5. Common Fetch Cycle RTLs

Every instruction begins with an identical fetch cycle that retrieves the instruction from memory and decodes the target address. The fetch cycle occupies timing states T0, T1, and T2. Execution commences at T3.

Time	RTL (Register Transfer)	Active Control Signals
T0	AR ← PC	LD AR = 1, BUS = PC
T1	IR ← M[AR], PC ← PC + 1	DR LD = 1, INC_PC = 1, read = 1, BUS = Memory
T2	AR ← IR[11:4]	LD AR = 1, BUS = IR address field

6. Execution RTLs Starting at T3

The following tables detail the register-transfer operations and control-signal assertions for each supported instruction, beginning at T3. The sequence counter (SC) is cleared at the conclusion of each instruction to return control to the fetch cycle (T0).

6.1 XOR Instruction

Time	RTL (Register Transfer)	End of Instruction
T3	DR ← M[AR]	
T4	AC ← AC XOR DR	SC ← 0

6.2 ADD Instruction

Time	RTL (Register Transfer)	End of Instruction
T3	DR ← M[AR]	
T4	AC ← AC + DR	SC ← 0

6.3 LDA Instruction

Time	RTL (Register Transfer)	End of Instruction
T3	DR ← M[AR]	
T4	AC ← DR	SC ← 0

6.4 STA Instruction

Time	RTL (Register Transfer)	End of Instruction
T3	M[AR] ← AC	SC ← 0

6.5 BUN Instruction

Time	RTL (Register Transfer)	End of Instruction
T3	PC ← AR	SC ← 0

6.6 DIV Instruction

Time	RTL (Register Transfer)	End of Instruction
T3	DR ← M[AR]	
T4	AC ← AC / DR	SC ← 0

6.7 ISZ Instruction

Time	RTL (Register Transfer)	End of Instruction
T3	DR ← M[AR]	
T4	DR ← DR + 1	
T5	M[AR] ← DR	
T6	If DR = 0 then PC ← PC + 1	SC ← 0

7. Register Control Logic Expressions

Let Z denote the zero flag produced after the ISZ increment, where Z = 1 when DR = 0x0000 after the T4 increment of ISZ. The plus sign (+) denotes logical OR, and juxtaposition (implicit multiplication) denotes logical AND. All clear signals for counter-based registers (SC, PC) are passed through a D flip-flop to

enforce synchronous clearing, as detailed in Section 11.

Control Signal	Logical Expression	Purpose
AR_LD	T0 + T2	Load AR from PC (T0) or from IR address field (T2)
IR_LD	T1	Load IR from memory during fetch
PC_INC	$T1 + D_6 \cdot T6 \cdot Z$	Increment PC during fetch (T1) or skip next instruction after ISZ zero result
PC_LD	$D_4 \cdot T3$	Load PC from AR for BUN branch
DR_LD	$(D_0+D_1+D_2+D_5+D_6) \cdot T3$	Load DR from memory for XOR, ADD, LDA, DIV, ISZ
DR_INC	$D_6 \cdot T4$	Increment DR during ISZ execution
AC_LD	$(D_0+D_1+D_2+D_5) \cdot T4$	Load AC from ALU result or DR
SC_CLR	$D_3 \cdot T3 + D_4 \cdot T3 + (D_0+D_1+D_2+D_5) \cdot T4 + D_6 \cdot T6$	Clear sequence counter at end of each instruction
SC_INC	NOT(SC_CLR)	Increment SC every clock while not clearing

8. Memory Control Logic Expressions

Memory read and write operations are gated by the timing signals and decoded instruction types. Read occurs during the fetch cycle and during T3 of memory-referencing instructions. Write occurs during store operations (STA) and the ISZ write-back phase.

Signal	Logical Expression	Description
MEM_READ	$T1 + (D_0+D_1+D_2+D_5+D_6) \cdot T3$	Fetch instruction (T1) or load DR from memory (T3 for XOR, ADD, LDA, DIV, ISZ)
MEM_WRITE	$D_3 \cdot T3 + D_6 \cdot T5$	Store AC to memory (STA at T3) or write incremented DR back (ISZ at T5)

9. BUS Encoder Input Logic

The common data bus is driven by a multiplexer (encoder) that selects among multiple register and memory sources. Each expression below corresponds to the select line for the indicated bus source. Only one source is active at any given time to avoid bus contention.

BUS Source	Encoder Select Expression	Used For
BUS_PC	T0	AR ← PC during fetch
BUS_MEM	$T1 + (D_0+D_1+D_2+D_5+D_6) \cdot T3$	IR ← M[AR] (T1), DR ← M[AR] (T3)
BUS_IR_ADDR	T2	AR ← IR[11:4] (address decode)
BUS_AC	$D_3 \cdot T3$	M[AR] ← AC for STA
BUS_AR	$D_4 \cdot T3$	PC ← AR for BUN
BUS_DR	$D_2 \cdot T4 + D_6 \cdot T5$	AC ← DR for LDA (T4), M[AR] ← DR for ISZ (T5)

10. ALU Select Logic

The Arithmetic Logic Unit (ALU) is implemented as a dedicated subcircuit. It receives AC and DR as primary inputs and produces output via an 8-to-1 multiplexer controlled by a 3-bit select line (C2C1C0). The ALU supports the following operations driven by the control unit. All unused ALU select inputs are tied to logic zero. The ALU output is routed to the destination register (AC or DR) via the internal datapath.

ALU Operation	Select Expression	Result Destination
ALU_XOR	$D_0 \cdot T4$	AC ← AC XOR DR
ALU_ADD	$D_1 \cdot T4$	AC ← AC + DR
ALU_PASS_DR	$D_2 \cdot T4$	AC ← DR (pass-through)
ALU_DIV	$D_5 \cdot T4$	AC ← AC / DR (integer division)
ALU_INC_DR	$D_6 \cdot T4$	DR ← DR + 1 (ISZ increment)

11. Synchronous Clear Implementation

Logisim implements counter-register clear signals asynchronously, causing immediate reset without waiting for the clock edge. To prevent premature clearing and ensure proper synchronous behaviour, all CLR signals for counter-based registers (particularly the Sequence Counter and PC) are routed through a D flip-flop. The CLR signal is connected to the D input of a D-FF, while the system clock drives the flip-flop's clock input. The Q output of the flip-flop then feeds the register's CLR pin. This configuration guarantees that the clear operation is synchronised to the positive clock edge, maintaining correct timing throughout the instruction cycle. The IR register is implemented as a standard register (not a counter), so it does not require this treatment.

Implementation: SC_CLR → D-FF (D input) → SC_CLR_SYNC (Q output) → SC.CLR pin. The same structure is applied to PC_LD and any other counter control signals requiring edge-triggered behaviour.

12. Logisim Circuit Implementation Overview

The complete basic computer was implemented in Logisim Evolution as a hierarchical circuit with two subcircuits: **main** and **ALU**. The design is partitioned into functional blocks connected via the common bus architecture.

Datapath: The central bus multiplexes six sources (PC, Memory, IR-address, AC, AR, DR) via an 8-to-1 multiplexer (select=3, 16-bit wide) into the destination registers. Tunnels are extensively used to reduce wire congestion and improve readability.

Registers: PC and AR are implemented as Counters (default width). DR, AC, and TR are 16-bit Counters (max=0xFFFF). IR is a 16-bit Register. SC is a 3-bit Counter (max=6, corresponding to T0–T6).

Decoders: A 4-to-16 opcode decoder (select=4) generates D_0–D_6. A 3-to-8 timing decoder (select=3) produces T0–T6 from the SC output.

Memory: A single RAM component (dataWidth=16, separate bus) stores both program instructions and data variables. Memory contents are saved in the Logisim raw hex format and loaded automatically upon circuit opening.

Control Unit: Combinational logic (AND-OR networks) derives all control signals from the decoded opcode and timing bits. A 16-bit Comparator generates the DR_ZERO flag used for the ISZ skip condition.

ALU: The ALU subcircuit receives AC and DR as inputs and is controlled by a 3-bit select line (C2C1C0) driving an 8-to-1 multiplexer. It contains an Adder, XOR Gate, Divider, NOT Gate, Subtractor, and Multiplier (all 16-bit), with output routed back to the main datapath.

Monitoring: Hex digit displays are attached to all major registers (PC, AC, AR, DR, IR, SC, TR) and the memory output bus for real-time debugging and verification.

13. Expected Clock Trace — First Program Loop

The following trace verifies the first complete iteration of the program, from fetch of LDA A through the BUN LOP branch back to address 0x0000. All values are shown in hexadecimal. The trace confirms correct datapath operation and control sequencing.

Instruction	Clock	Expected Value / Change	Notes
LDA A	T0	AR = 0000	Fetch: AR ← PC
	T1	IR = 20B0, PC = 0001	Fetch: IR ← M[AR], PC++
	T2	AR = 000B	Decode: AR ← IR[11:4] = 0x0B
	T3	DR = 000F	DR ← M[0B] = A = 15
	T4	AC = 000F	AC ← DR; SC ← 0
XOR B	T0	AR = 0001	Fetch next instruction
	T1	IR = 00C0, PC = 0002	
	T2	AR = 000C	Address of B = 0x0C
	T3	DR = 0005	DR ← M[0C] = B = 5
	T4	AC = 000A	AC = 0x0F XOR 0x05 = 0x0A (10); SC ← 0
ADD A	T0	AR = 0002	
	T1	IR = 10B0, PC = 0003	
	T2	AR = 000B	Address of A
	T3	DR = 000F	DR ← M[0B] = A = 15
	T4	AC = 0019	AC = 0x0A + 0x0F = 0x19 (25); SC ← 0
STA A	T0	AR = 0003	
	T1	IR = 30B0, PC = 0004	
	T2	AR = 000B	Address of A
	T3	M(000B) = 0019	Store AC to A; SC ← 0
LDA A	T0	AR = 0004	
	T1	IR = 20B0, PC = 0005	
	T2	AR = 000B	
	T3	DR = 0019	DR ← updated A = 25
	T4	AC = 0019	AC ← DR; SC ← 0
DIV B	T0	AR = 0005	
	T1	IR = 50C0, PC = 0006	

Instruction	Clock	Expected Value / Change	Notes
	T2	AR = 000C	Address of B
	T3	DR = 0005	DR ← M[0C] = B = 5
	T4	AC = 0005	AC = 0x19 / 0x05 = 0x05 (25/5=5); SC ← 0
ADD C	T0	AR = 0006	
	T1	IR = 10D0, PC = 0007	
	T2	AR = 000D	Address of C
	T3	DR = 0000	DR ← M[0D] = C = 0
	T4	AC = 0005	AC = 0x05 + 0x00 = 0x05; SC ← 0
STA C	T0	AR = 0007	
	T1	IR = 30D0, PC = 0008	
	T2	AR = 000D	Address of C
	T3	M(000D) = 0005	Store AC to C; SC ← 0
ISZ i	T0	AR = 0008	
	T1	IR = 60E0, PC = 0009	
	T2	AR = 000E	Address of i
	T3	DR = FFFD	DR ← M[0E] = i = –3
	T4	DR = FFFE	DR ← DR + 1 = –2
	T5	M(000E) = FFFE	Write back incremented i
	T6	PC unchanged	DR ≠ 0, so no skip; SC ← 0
BUN 0	T0	AR = 0009	
	T1	IR = 4000, PC = 000A	
	T2	AR = 0000	Branch target address
	T3	PC = 0000	PC ← AR; SC ← 0. Loop repeats.

14. Loop Result Summary

The program executes three complete loop iterations before the ISZ instruction causes a skip of the BUN branch, terminating the loop. The following table summarises the memory contents at the loop checkpoints. Note: 0x0019 XOR 0x0005 = 0x001C (28), and 0x0035 XOR 0x0005 = 0x0030 (48), confirming the arithmetic.

Checkpoint	A (M[0B])	C (M[0D])	i (M[0E])	Decimal Notes
After 1st loop	0019	0005	FFFE	A = 25, C = 5, i = –2
After 2nd loop	0035	000F	FFFF	A = 53, C = 15, i = –1
After 3rd loop	0065	0023	0000	A = 101, C = 35, i = 0. ISZ skips BUN.

15. Final Output

Upon termination of the third loop iteration, the ISZ instruction detects that the loop counter i has reached zero, asserts the skip condition (PC_INC), and the subsequent BUN instruction is bypassed. The program halts with the following final values:

Variable	Final Hex Value	Final Decimal Value
A (M[0B])	0065	101
C (M[0D])	0023	35
i (M[0E])	0000	0

16. Conclusion

The control unit, datapath, and memory system were successfully designed and verified to execute the specified program. The RTLs and control signal expressions correctly implement the fetch-decode-execute cycle for all seven required instructions: XOR, ADD, LDA, STA, BUN, DIV, and ISZ. The synchronous clear mechanism using D flip-flops ensures reliable counter operation within Logisim. The program executes exactly three loop iterations, producing the required final outputs: A = 101 (0x0065) and C = 35 (0x0023), with the loop counter i = 0. All unused control signals are held at logic zero, maintaining datapath stability throughout execution.

Appendix A: Memory Contents Text File Format

For Logisim memory persistence, the RAM contents are saved in the following text format (address-value pairs in hexadecimal, one per line). This file can be loaded via right-click on the RAM component → Load Image.

```
v2.0 raw 20B0 00C0 10B0 30B0 20B0 50C0 10D0 30D0 60E0 4000 0000 000F 0005 0000 FFFD
```

Appendix B: Deliverables Checklist

■ Logisim circuit file (.circ) with complete datapath, control unit, ALU, and memory.

■ This report containing RTLs for T0–T2 (fetch) and T3+ (execution) for all instructions.

■ Logical expressions for all register LD, INC, CLR signals and the sequence counter.

■ Logical expressions for memory READ and WRITE control.

■ Logical expressions for BUS encoder select lines and ALU operation select lines.

■ Memory contents text file (hex format for Logisim RAM loading).

■ Text file with group member names, IDs, tutorial numbers, and email addresses.

Appendix C: Team Members

Name	ID	Tutorial	Email
Habiba Abdou Hussein	64-1442	T-6	habiba.abdou@student.guc.edu.eg
Baraa Khaled Mohamed	64-12893	T-6	baraa.gad@student.guc.edu.eg
Hady Weaam	61-13384	T-7	hady.weaam@student.guc.edu.eg
Ahmed Hagras	64-8836	T-8	ahmed.hagras@student.guc.edu.eg
Hamza Mohamed Abdelghany	64-13596	T-8	hamza.abdelghany@student.guc.edu.eg
Amr Elsayed Elsayed	58-13562	T-23	Amr.elsayedelsayed@student.guc.edu.eg